0743. Network Delay Time

m | Category: Advanced Graphs | LeetCode: https://leetcode.com/problems/netw

Problem

- Directed weighted edges times[i] = (u, v, w). A signal starts at node k. Return the time for all n nodes to receive it, or -1.

Intuition

- Single-source shortest paths with non-negative weights: Dijkstra. The answer is the largest shortest distance.

Brute Force

- Bellman-Ford: O(V E).

Optimal Approach

- Adjacency list; min-heap of (dist, node) starting with (0, k).
- Pop; if already finalised skip; else record dist and push neighbours with dist + w.
- If all n nodes are finalised, return the max distance; else -1.

Worked Example

```
times [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2.

pop (0,2)  -> push (1,1) (1,3)
pop (1,1)  final ; pop (1,3) -> push (2,4)
pop (2,4)  final
max = 2
```

Java Solution

```java
public class NetworkDelayTime {
    public int networkDelayTime(int[][] times, int n, int k) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i <= n; i++) adj.add(new ArrayList<>());
        for (int[] t : times) adj.get(t[0]).add(new int[]{t[1], t[2]});
        int[] dist = new int[n + 1];
        Arrays.fill(dist, -1);
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        pq.add(new int[]{0, k});
        int done = 0, max = 0;
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int d = cur[0], u = cur[1];
            if (dist[u] != -1) continue;
            dist[u] = d;
            done++;
            max = Math.max(max, d);
            for (int[] e : adj.get(u)) if (dist[e[0]] == -1) pq.add(new int[]{d + e[1], e[0]});
        }
        return done == n ? max : -1;
    }
}
```

Complexity

- Time: O(E log V). Space: O(V + E).

Pitfalls

- Lazy deletion: skip stale heap entries for nodes already finalised.